Surveillance d'une ligne de production : simulation TinyOS / Cooja

Dix cartes simulées aux positions exactes du plan d'implantation, un réseau radio multi-saut (CTP), des capteurs de température, de comptage et de présence, et une passerelle qui produit les messages MQTT du cahier des charges. Chaque exigence vérifiable a son test automatique ; toutes les valeurs de ce rapport sont extraites des journaux de ces tests.

17 / 18
exigences vérifiées conformes (1 non conforme : ENF5 ; 1 non mesurable)

33 ms
latence moyenne capteur → passerelle (T6)

100,0 %
taux de livraison, 10 nœuds, 20 min

100 %
des rapports d'1 h de coupure récupérés

1. Objectifs et correspondance réel ↔ simulation

Le système réel (architecture du groupe) repose sur des ESP32 en WiFi et un Raspberry Pi (Mosquitto, Node-RED, InfluxDB, Grafana). Cooja n'émule ni l'ESP32 ni le WiFi : il émule, instruction par instruction, des cartes de réseau de capteurs. On garde donc ce qui fait la difficulté du projet — la **géographie** de l'usine, les **rôles**, les **exigences temporelles** — et on remplace le matériel par son équivalent « réseau de capteurs ».

DANS L'USINE	DANS LA SIMULATION	POURQUOI / CONSÉQUENCE
ESP32 + capteur	Carte Sky (TelosB, MSP430 + radio CC2420), programmée en nesC / TinyOS	Seule carte que TinyOS compile et que Cooja émule ; 10 Ko de RAM, 48 Ko de ROM
WiFi 2,4 GHz, point d'accès	IEEE 802.15.4 (250 kbit/s), modèle radio UDGM, portée 20 m	Plus courte portée que le WiFi (30 m retenus) : impose le multi-saut, comme un vrai réseau de capteurs
Raspberry Pi (passerelle + serveur)	Nœud 1 : racine CTP, écrit sur son port série une ligne « topic + JSON » par message	Le pont série → MQTT est la seule partie non simulée
DS18B20 / thermocouple K	Modèle thermique (montée en régime, dérive, bruit ± 0,3 °C)	Scénario de surchauffe injectable (commande H)
Barrière infrarouge FC-51	Arrivées de pièces aléatoires (2 à 6 s), compteur en flash	100 pièces injectables (commande +)
PIR HC-SR501 + buzzer	Détections aléatoires de jour, présence injectable (P), buzzer = LED bleue	Horloge d'usine accélérée : 1 s = 1 min, départ 17h00
Coupure réseau	La passerelle est éloignée à 1 km pendant 1 h	Coupure de la liaison sans arrêter la passerelle
Nœud débranché	Nœud retiré de la simulation (sim.removeMote)	Panne franche, sans prévenir les voisins

> **Limites de la correspondance.** Les latences mesurées sont celles d'un réseau 802.15.4 non chargé, sans le trajet passerelle → broker → Node-RED → notification (budget < 3 + 3 + 2 s dans ENF1, non simulé). L'énergie est celle de la radio CC2420, pas d'un ESP32. Le modèle UDGM est idéal (disque de portée, pas d'obstacle ni d'évanouissement) ; le test T6b le dégrade volontairement. Les capteurs sont simulés : la précision de ± 1 °C (EF1) dépend du composant réel (DS18B20 : ± 0,5 °C), pas du réseau.

---

2. Architecture de la simulation

2.1 Quatre rôles, quatre programmes

Une application nesC par rôle, avec un dossier `commun/` partagé. Cooja charge donc quatre « types de cartes » ; chaque carte connaît son numéro (`TOS_NODE_ID` , écrit par Cooja) et en déduit, grâce à `Plan.h` généré depuis le plan, son repère, sa machine et son seuil.

NŒUDS	PROGRAMME	CE QU'IL FAIT
1 (GW)	`passerelle`	Racine CTP ; vérifie l'authentification ; écrit topic + JSON ; alertes ; statistiques ; diffuse les acquittements
2–4 (T1–T3)	`thermique`	Lecture toutes les 2 s, rapport toutes les 30 s, alerte immédiate au-delà du seuil de SA machine
5–7 (C1–C3)	`comptage`	+1 à chaque pièce (sauvegarde en flash), rapport toutes les 30 s : cumul + cadence
8–10 (P1–P3)	`intrusion`	Suit l'heure de l'usine ; de nuit, toute présence → alerte immédiate + buzzer ; rapport de vie toutes les 30 s

2.2 Le message radio

Le JSON du cahier des charges est trop gros et trop coûteux à émettre sur une radio basse consommation. Les capteurs envoient une structure binaire de 23 octets ; c'est la passerelle qui fabrique le JSON. Un exemple de ligne réellement écrite par la passerelle (test T6) :

```
RX src=4 seq=1 sauts=1 parent=1 age=4 diff=0 | factory/machines/MACHINE_03
{"node_id":"MACHINE_03","timestamp":1792512010,"temperature_c":61.5,"production_count":2,"status":"OK"}
```

CHAMP	TAILLE	CONTENU
type, etat	1 + 1 octet	température / comptage / présence ; OK, ALERTE, FIN_ALERTE, VEILLE, ARME, INTRUSION
source, seq	2 + 2	numéro du nœud et numéro de rapport (sert au taux de livraison et aux acquittements)
age_ms	4	temps passé dans le tampon : la passerelle en déduit l'horodatage réel de la mesure
valeur, compteur	2 + 4	dixièmes de °C · cadence et cumul de pièces · heure d'usine et nombre de détections
parent, etx	2 + 2	le premier relais et le coût de route CTP : permet de dessiner l'arbre
differe	1	1 si le rapport a attendu une coupure
mac	2	code d'authentification calculé avec la clé du réseau (ENF6)

2.3 Routage multi-saut (CTP) et acquittements de bout en bout (Drip)

```
capteur ──PUB──> tampon ──CTP (saut par saut, accusé radio à chaque saut)──> passerelle ──> ligne M
   ^                                                                                      |
   └──────────────── Drip : « nœud 8, j'ai tout reçu jusqu'au n° 41 » (toutes les 30 s) ┘
```

CTP (Collection Tree Protocol, `tos/lib/net/ctp`) construit seul un arbre dont la passerelle est la racine. Chaque nœud estime la qualité de ses liens (ETX : nombre moyen d'émissions pour réussir un envoi) et choisit le voisin qui minimise le coût total jusqu'à la racine. À 20 m de portée, P1 (22,7 m), P2 (26,2 m) et P3 (20,8 m) ne voient pas la passerelle : leurs rapports passent par un relais.

Drip (`tos/lib/net/drip`) fait le chemin inverse : il propage une valeur de la passerelle vers tous les nœuds. On l'utilise pour un **acquittement de bout en bout** — l'équivalent du QoS 1 de MQTT — car l'accusé de réception de CTP ne prouve que la réception par le voisin (voir § 5.5 : sans lui, une coupure faisait perdre 100 % des données). L'acquittement est **signé** par la passerelle avec la clé du réseau : un acquittement altéré ou falsifié ferait effacer aux capteurs des données jamais reçues (voir § 5.6).

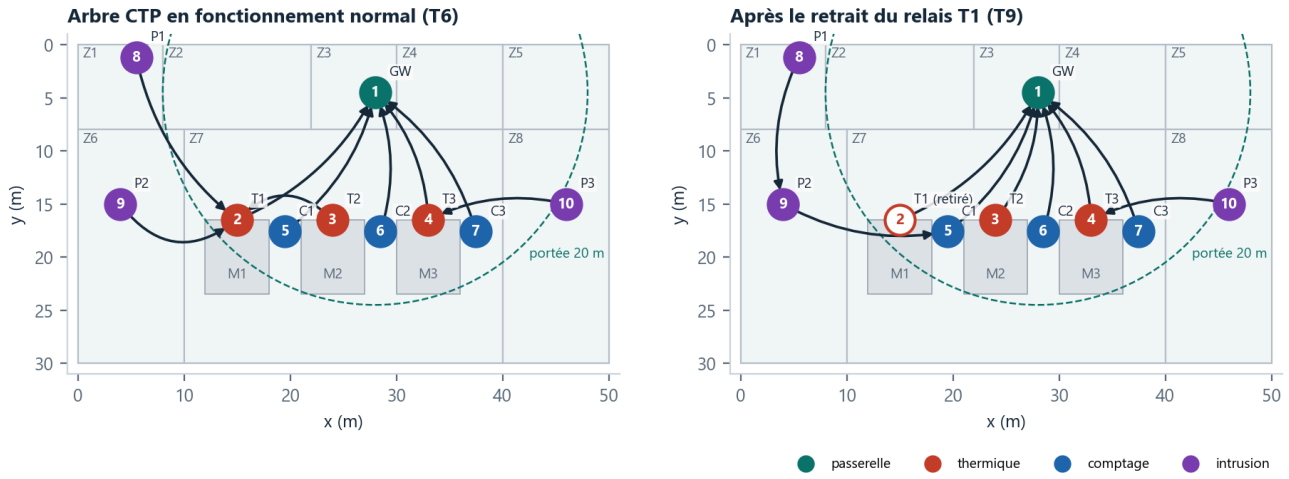


Figure 1 — Positions du plan (mètres) et arbre CTP mesuré (flèche = « je confie mes paquets à »). Le cercle pointillé est la portée de 20 m autour de la passerelle. À droite, après le retrait de T1, P1 passe par P2.

---

3. Le code, pas à pas

TinyOS est piloté par des **événements** : pas de boucle infinie, on écrit ce qu'il faut faire quand « la carte a démarré », « la minuterie a sonné », « un paquet est arrivé ». Un composant nesC a deux fichiers : une **configuration** (le câblage, avec des flèches `->`) et un **module** (le comportement).

3.1 Câbler une application : le nœud thermique

```
configuration ThermiqueAppC {}
implementation {
  components MainC, LedsC, RandomC, ConsoleC, PublicationC;
  components ThermiqueC as App;
  components new TimerMilliC() as Lecture;
  components new TimerMilliC() as Rapport;

  App.Boot        -> MainC;
  App.Leds        -> LedsC;
  App.Random      -> RandomC;
  App.Console     -> ConsoleC;
  App.Publication -> PublicationC;
  App.Lecture     -> Lecture;
  App.Rapport     -> Rapport;
}
```

thermique/ThermiqueAppC.nc, ligne 7

Le nœud thermique ne connaît ni la radio ni CTP : il utilise `PublicationC`, notre service commun, et `ConsoleC` pour écrire du texte lisible dans Cooja.

3.2 Lire le capteur et alerter tout de suite

```
  event void Lecture.fired() {
    if (surchauffe) {
      temp = seuil + 50 + aleatoire(3);        /* la sonde lit seuil + 5 °C */
    } else {
      derive += aleatoire(2);
      if (derive > 30) derive = 30;
      if (derive < -30) derive = -30;
      temp += (consigne + derive - temp) / 20;
      temp += aleatoire(3);                    /* bruit ±0,3 °C */
    }

    if (temp > seuil && !enAlerte) {
      enAlerte = TRUE;
      call Leds.led2On();                      /* LED bleue = machine en alerte */
      call Console.texte("DEPASSEMENT temp="); call Console.dixiemes(temp);
      call Console.texte(" seuil=");           call Console.dixiemes(seuil);
      call Console.car('\n');
      call Publication.publier(TYPE_TEMPERATURE, ETAT_ALERTE, temp, 0);
    } else if (enAlerte && temp <= seuil - HYSTERESIS_DIXIEMES) {
      enAlerte = FALSE;
      call Leds.led2Off();
      call Console.texte("RETOUR_NORMAL temp="); call Console.dixiemes(temp);
      call Console.car('\n');
      call Publication.publier(TYPE_TEMPERATURE, ETAT_FIN_ALERTE, temp, 0);
    }
  }

  /* ------------------------------------------------------------------ rapport périodique */
```

thermique/ThermiqueC.nc, ligne 71

La lecture locale toutes les 2 s ne coûte rien en radio. Sans elle, une surchauffe pourrait attendre jusqu'à 30 s le rapport suivant, ce qui violerait ENF1.

3.3 Ne rien perdre : tampon et acquittements

```
  event void Acquit.changed() {
    const acquit_t *a = call Acquit.get();
    uint16_t ok = (TOS_NODE_ID <= MAX_ID) ? a->dernier[TOS_NODE_ID] : 0;
    uint8_t liberes = 0;

    /* Acquittement non signé par la passerelle (paquet altéré ou falsifié) : on l'ignore,
       sinon on effacerait des rapports que la passerelle n'a jamais reçus. */
#ifndef INTRUS   /* le nœud intrus du test ENF6 ne respecte pas le protocole : il émet quand même
*/
    if (macAcquit(a, CLE_RESEAU) != a->mac) {
      call Console.texte("ACQUIT_REJETE code d'authentification invalide\n");
      return;
    }
#endif
    dernierAcquit = maintenant();
    call Console.texte("ACQUIT battement="); call Console.nombre(a->battement);
    call Console.texte(" ok=");              call Console.nombre(ok);
    call Console.car('\n');
    if (!enLigne) {
      enLigne = TRUE;
      call Console.texte("EN_LIGNE acquittements recus, ");
      call Console.nombre(nb);
      call Console.texte(" rapport(s) a envoyer\n");
    }
    /* On oublie tous les rapports que la passerelle a bien reçus */
    while (nb > 0 && entree(0)->seq <= ok && !(occupe && envoyes == 0)) {
      debut = (debut + 1) % CAPACITE_TAMPON;
      nb--;
      if (envoyes > 0) envoyes--;
      liberes++;
    }
```

commun/PublicationP.nc, ligne 161

3.4 La passerelle : authentifier, dédoublonner, publier, alerter

```
    /* 1. Authentification : on recalcule le code avec NOTRE clé et on compare */
    if (calculerMac(r, CLE_RESEAU) != r->mac) {
      call Console.texte("REJET_AUTH origine=");
      call Console.nombre(call CtpPacket.getOrigin(msg));
      call Console.texte(" seq=");  call Console.nombre(r->seq);
      call Console.texte(" (code d'authentification invalide)\n");
      return msg;
    }
    /* Clé correcte, mais nœud absent du plan ou qui se fait passer pour un autre */
    if (r->source > MAX_ID || r->source != call CtpPacket.getOrigin(msg)) {
      call Console.texte("REJET_ORIGINE origine=");
      call Console.nombre(call CtpPacket.getOrigin(msg));
      call Console.texte(" source annoncee="); call Console.nombre(r->source);
      call Console.car('\n');
      return msg;
    }

    src = r->source;
    s = &suivi[src];
    /* THL (« time has lived ») : CTP l'augmente de 1 à chaque réception, y compris
       à la racine (CtpForwardingEngineP). C'est donc directement le nombre de sauts. */
```

passerelle/PasserelleC.nc, ligne 185

3.5 Le compteur qui survit au redémarrage

```
    /* IMPORTANT : pendant l'effacement et l'écriture, le contrôleur de flash bloque le CPU.
       Une interruption à ce moment fait planter le MSP430 (mspsim le reproduit :
       « Got interrupt while flash controller blocks CPU »). Le module d'origine servait
       au chargeur d'amorçage, sans interruptions ; ici on les MASQUE avec atomic. */
    atomic {
      FCTL2 = FWKEY + FSSEL1 + FN2;     /* horloge du contrôleur de flash */
      FCTL3 = FWKEY;                    /* déverrouillage */
      FCTL1 = FWKEY + ERASE;            /* effacement du segment de destination */
```

commun/FlashInterneC.nc, ligne 49

Le compteur est écrit dans la mémoire flash interne du MSP430 à chaque pièce, et relu au démarrage. mspsim reproduit fidèlement une contrainte du vrai matériel : une interruption pendant l'écriture fait planter le microcontrôleur. D'où le bloc `atomic`.

3.6 Un test automatique

```
TIMEOUT(260000, terminer());
var tInj = {}, tDetect = {}, tAlerte = {}, alerteM3Avant = false, finM2 = false, statusAlerte =
false;
planifier(120000, function () { tInj[3] = maintenant(); commander(3, "H"); });
planifier(160000, function () { commander(3, "N"); });
planifier(200000, function () { tInj[4] = maintenant(); commander(4, "H"); });
planifier(250000, function () {
  var l2 = tAlerte[3] - tInj[3], l3 = tAlerte[4] - tInj[4];
  mesure("T2_latence_injection_alerte_M2", l2, "ms");
  mesure("T2_latence_detection_alerte_M2", tAlerte[3] - tDetect[3], "ms");
  mesure("T2_latence_injection_alerte_M3", l3, "ms");
  verdict("T2", tAlerte[3] !== undefined && l2 < 10000,
          "M2 a 95 C : alerte " + (l2 / 1000).toFixed(2) + " s apres l'injection (exigence < 10 s,
cible 5 s)");
  verdict("T2_json_status", statusAlerte, "JSON MACHINE_02 avec status ALERTE publie");
  verdict("T2_seuil_par_machine", !alerteM3Avant && tAlerte[4] !== undefined && l3 < 10000,
          "M3 a ~185 C sans alerte (seuil 220 C), puis alerte a 225 C en " + (l3 /
1000).toFixed(2) + " s");
  verdict("T2_fin_alerte", finM2, "FIN_ALERTE SURCHAUFFE MACHINE_02 apres retour a la normale");
  terminer();
});
while (true) {
  YIELD();
  if (executerAction()) continue;
  noter();
```

scenarios/tests/T2_surchauffe.js, ligne 8

Le script, exécuté par le plugin ScriptRunner de Cooja, reçoit chaque ligne écrite par chaque carte, injecte des événements sur les ports série (`commander(3, "H")`), mesure les délais avec l'horloge de Cooja (commune à tous les nœuds) et écrit ses verdicts.

---

4. Les scénarios de test

TEST	DURÉE SIMULÉE	SCÉNARIO
T1	10 min	fonctionnement normal ; on mesure l'écart entre rapports de chaque capteur
T2	4 min	t = 120 s : sonde de M2 à 95 °C ; t = 160 s : retour normal ; t = 200 s : four M3 à 225 °C
T4	4 min	C1 : arrêt des arrivées, remise à zéro, 100 pièces ; C2 : redémarrage forcé à t = 150 s
T5	16 min	présences à 17h30 (P2), 19h00 (P1, P2, P3) et 08h30 le lendemain (P3)

TEST	DURÉE SIMULÉE	SCÉNARIO
T6 / T6b	20 min	fonctionnement normal, 10 nœuds ; T6b : 80 % de réussite radio en limite de portée
T7	75 min	t = 5 min : passerelle hors de portée pendant 1 h ; 10 min d'observation après le retour
T9	10 min	t = 5 min : retrait du nœud T1, relais de P1
ENF6	5 min	un 11e nœud, programmé avec une autre clé, posé près de T2
Énergie	4 × 15 min	radio toujours allumée, puis écoute basse consommation (LPL) à 512, 1024, 2048 ms

5. Résultats

5.1 Réseau : latence, sauts, taux de livraison (T6, T6b)

NŒUD	X ; Y (M)	PARENT CTP	SAUTS	LATENCE MOY. (MS)	MAX (MS)	PDR T6	PDR T6B
T1 (2)	15 ; 16.5	1 (GW)	1	29,0	33	100,0 %	100,0 %
T2 (3)	24 ; 16.5	2 (T1)	2	40,0	48	100,0 %	100,0 %
T3 (4)	33 ; 16.5	1 (GW)	1	28,5	34	100,0 %	100,0 %
C1 (5)	19.5 ; 17.6	1 (GW)	1	28,0	33	100,0 %	100,0 %
C2 (6)	28.5 ; 17.6	1 (GW)	1	27,9	33	100,0 %	100,0 %
C3 (7)	37.5 ; 17.6	1 (GW)	1	27,4	32	100,0 %	100,0 %
P1 (8)	5.5 ; 1.2	2 (T1)	2	38,7	45	100,0 %	100,0 %
P2 (9)	4 ; 15	2 (T1)	2	39,9	56	100,0 %	100,0 %
P3 (10)	46 ; 15	4 (T3)	2	38,6	47	100,0 %	100,0 %

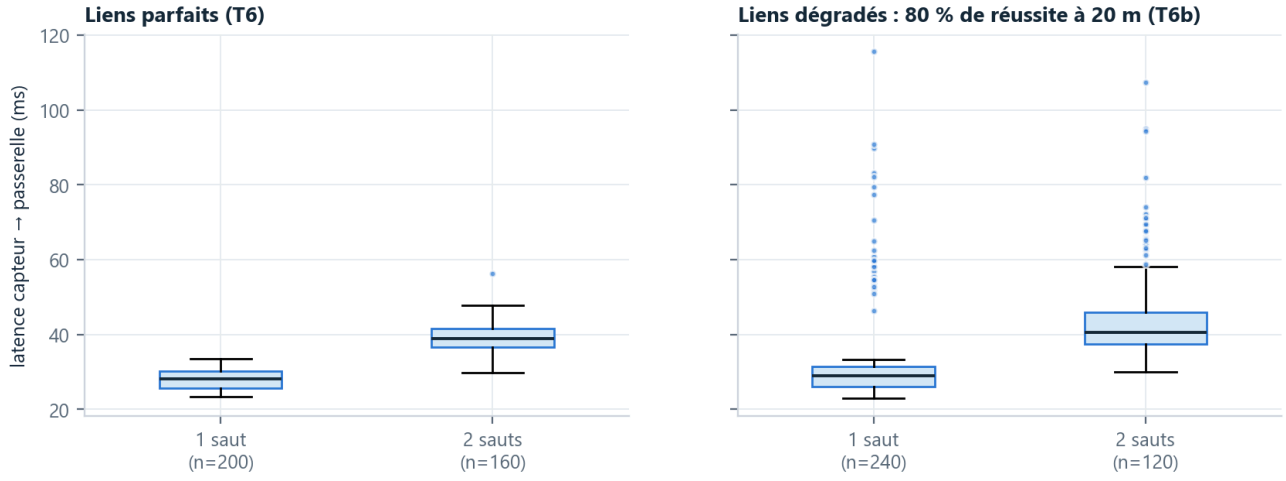


Figure 2 — Latence capteur → passerelle de chaque rapport, par nombre de sauts. Les liens dégradés provoquent des retransmissions (points isolés) mais aucune perte : CTP réémet jusqu'à 30 fois.

La latence mesurée inclut l'écriture de la ligne JSON sur le port série de la passerelle (≈ 11 ms à 115 200 bit/s) : la part radio est encore plus faible. On est très loin du budget de 1 s du cahier pour le trajet nœud → passerelle.

Observation : T2 (nœud 3, à 12,6 m du puits) passe par T1. CTP fige son choix au démarrage et ne change de parent que si le gain dépasse un seuil (hystérésis), pour éviter les oscillations de route. Le coût est d'environ 10 ms par rapport.

5.2 Alertes de surchauffe (T2)

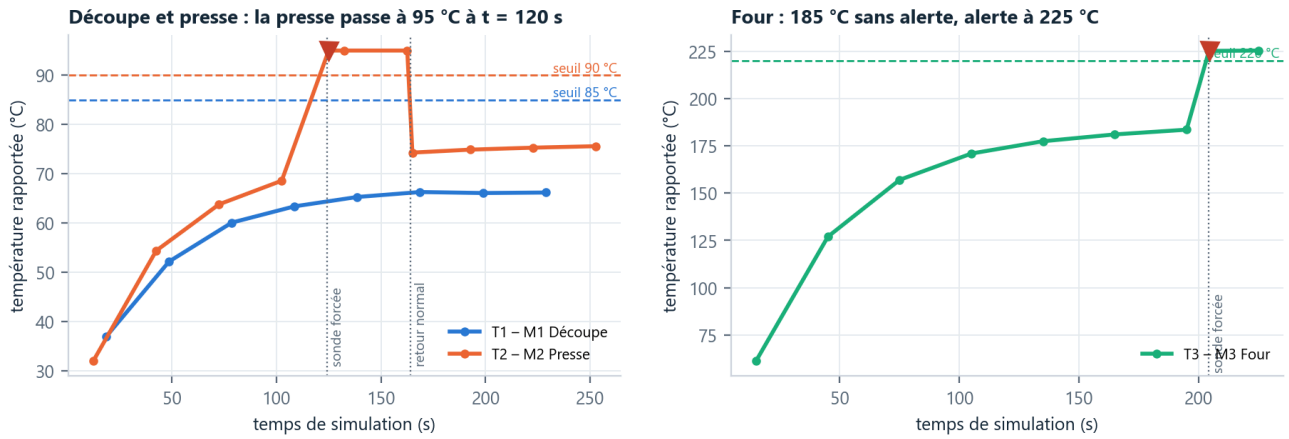


Figure 3 — Températures rapportées (points = rapports, triangles = alertes de la passerelle). Chaque machine a son seuil : le four tourne à 185 °C sans alerte.

De la mise à 95 °C à la ligne `ALERTE SURCHAUFFE MACHINE_02` : **0,88 s**, dont 51 ms entre la détection sur le nœud et l'alerte — le reste est l'attente de la lecture suivante (au plus 2 s). Cible de 5 s tenue.

5.3 Comptage et intrusion (T4, T5)

T4 : après 100 pièces, le compteur local vaut 100 et la passerelle publie `"production_count":100`. Le nœud C2, redémarré par son chien de garde à 40 pièces, relit 40 dans sa flash ; la passerelle détecte le redémarrage (numéro de rapport revenu à 1).

T5 : de nuit, l'alerte arrive en 48, 51 et 49 ms pour les zones 1, 2 et 3 (deux sauts chacune), avec le buzzer local. Les présences à 17h30 et à 08h30 ne déclenchent aucune alerte.

5.4 Panne d'un relais et reconvergence (T9, ENF8)

MESURE (DEPUIS LE RETRAIT DE T1)	VALEUR
P1 choisit un nouveau parent	18,2 s — s (P1 : parent 2 -> 9)
Premier rapport de P1 reçu par le nouveau chemin	47,3 s — s (premier rapport de P1 recu, 3 sauts)
La passerelle signale « NOEUD_MUET T1 »	84,5 s (seuil : 3 périodes + 5 s sans nouvelles)
Taux de livraison des 8 autres capteurs après la panne	100,0 %

CTP ne détecte la panne qu'en essayant d'envoyer : P1 découvre l'absence de T1 à son rapport suivant, épuise ses retransmissions, puis change de parent. Le rapport retardé n'est pas perdu : il reste dans le tampon jusqu'à son acquittement.

5.5 Coupure d'une heure (T7, ENF3)

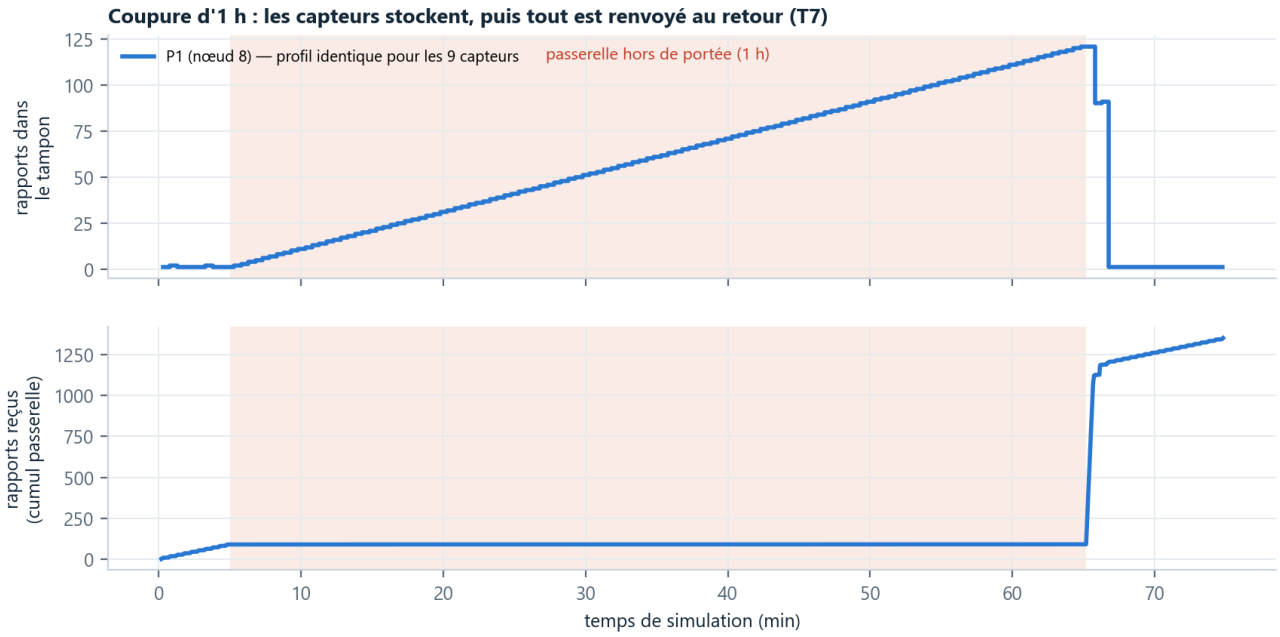


Figure 4 — En haut : rapports en attente dans le tampon de trois capteurs (1 par 30 s). En bas : cumul des rapports reçus par la passerelle ; la marche au retour est le vidage des tampons.

1080 rapports sur 1080 publiés pendant la coupure ont été reçus après le retour, tous marqués `diff=1`, en 114 s. Leur horodatage JSON est celui de la mesure, grâce au champ `age_ms` : écart maximal 3,6 s. Le cahier ne fixant pas de tolérance, nous avons retenu ± 5 s (1/6 de la période : une mesure ne peut pas être confondue avec sa voisine). L'écart restant est le temps passé dans les files des relais pendant le vidage, que `age_ms` ne compte pas ; une horloge commune (synchronisation) l'éliminerait.

> **Ce que ce test nous a appris.** La première version perdait **100 %** des données de la coupure. Quand la racine disparaît, aucun nœud ne le sait : chacun prend un voisin pour parent, le coût de route gonfle sans fin (« comptage à l'infini », ETX de 20 à 12 000) et les paquets tournent en boucle avant d'être jetés. L'accusé de réception de CTP était pourtant positif, puisque le voisin avait bien reçu. Quatre corrections, chacune validée par un nouvel essai : (1) un acquittement de bout en bout diffusé par la passerelle, et l'arrêt des émissions après 75 s sans acquittement ; (2) une balise de routage forcée par la racine toutes les 30 s, sans quoi les nœuds ne la retrouvaient pas à son retour (une racine stable espace ses balises jusqu'à plusieurs minutes) ; (3) au retour, les 9 capteurs vidaient leurs tampons en rafale et saturaient les files des relais : le vidage est cadencé à un rapport toutes les 250 ms ; (4) la passerelle « abandonnait » les trous de plus de 32 numéros et acquittait donc des rapports jamais reçus (57 % de pertes) : elle suit désormais une fenêtre de 160 numéros, plus grande que le tampon, et n'acquitte jamais un rapport qu'elle n'a pas reçu.

Capacité : le tampon garde 130 rapports, soit 65 min à 30 s. ENF3 demande 24 h, soit 2 880 rapports (≈ 58 Ko) : il faudrait la flash externe de 1 Mo de la carte (`LogStorageC`), ou la mémoire flash de l'ESP32 dans le prototype réel.

5.6 Énergie (ENF5)

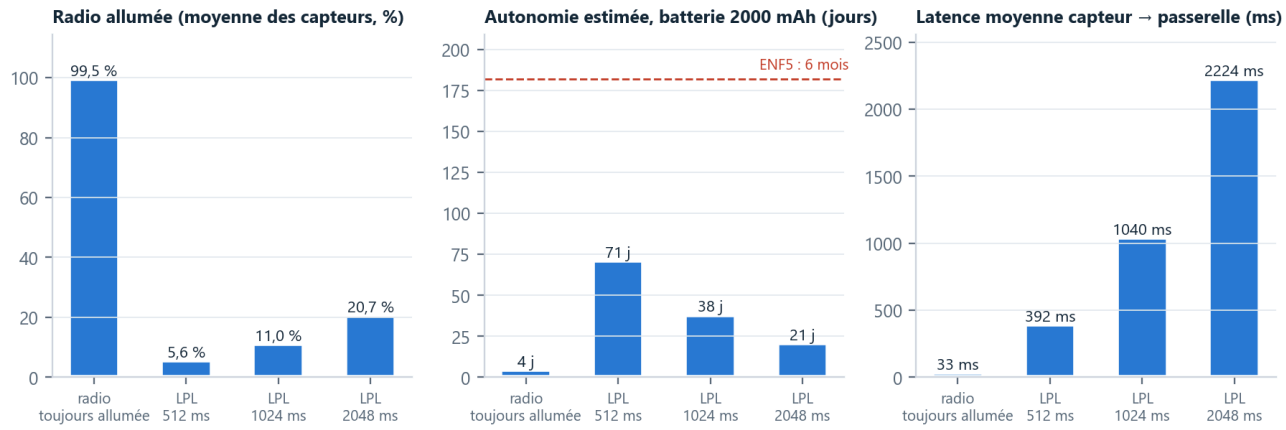


Figure 5 — Mesures PowerTracker (15 min, 10 nœuds). Autonomie estimée pour 2000 mAh avec 19 mA radio allumée et 0,1 mA pour le microcontrôleur (hypothèses, pas des mesures).

RADIO	LATENCE MOY. (MS)	95 % SOUS (MS)	MAX (MS)	LIVRAISON À TERME
toujours allumée	33	43	56	100,00 %
LPL 512 ms	392	1983	5072	100,00 %
LPL 1024 ms	1040	4479	23607	100,00 %
LPL 2048 ms	2224	9173	19800	100,00 %

NŒUD	TOUJOURS ALLUMÉE	LPL 512 MS	LPL 1024 MS	LPL 2048 MS
GW (1)	99,5 %	99,5 %	99,5 %	99,5 %
T1 (2)	99,5 %	5,1 %	10,3 %	19,2 %
T2 (3)	99,5 %	3,5 %	7,9 %	18,4 %
T3 (4)	99,5 %	3,8 %	6,5 %	16,5 %
C1 (5)	99,5 %	4,0 %	7,0 %	10,5 %
C2 (6)	99,5 %	4,0 %	6,4 %	15,3 %
C3 (7)	99,5 %	4,1 %	13,5 %	21,3 %
P1 (8)	99,5 %	11,1 %	13,6 %	29,3 %
P2 (9)	99,5 %	7,8 %	18,8 %	25,6 %
P3 (10)	99,5 %	7,3 %	15,0 %	30,6 %

Radio toujours allumée, la radio d'un capteur est active 99,5 % du temps : environ 4 jours sur 2000 mAh. C'est l'argument du cahier contre une écoute permanente (un ESP32 connecté en WiFi consomme encore davantage). L'écoute basse consommation (LPL) éteint la radio et la rallume brièvement à intervalle fixe ; pour joindre un voisin endormi, l'émetteur répète son paquet pendant tout un intervalle. Le meilleur réglage mesuré est **LPL 512 ms** : radio active 5,6 % du temps, soit environ **71 jours** — en deçà des 6 mois d'ENF5 avec ces hypothèses.

Latence : avec LPL 512 ms, la latence maximale reste sous les 10 s d'ENF1 (mais il faut 2,0 s pour que 95 % des rapports arrivent, au-delà du budget d'1 s prévu pour le trajet nœud → passerelle : l'alerte arrive à temps, en consommant une plus grosse part du budget). En revanche LPL 1024 ms (23,6 s), LPL 2048 ms (19,8 s) dépassent les 10 s : ces réglages sont incompatibles avec ENF1.

Résultat contre-intuitif : espacer les réveils (1024 puis 2048 ms) *augmente* la consommation. Notre réseau diffuse beaucoup — balises de routage CTP, acquittements Drip répétés de proche en proche — et en LPL chaque diffusion dure tout un intervalle de réveil. Au-delà de 512 ms, le coût des émissions dépasse l'économie d'écoute, et le canal se sature (latence en secondes, plus de collisions). Pour viser 6 mois, il faut donc réduire les diffusions plutôt qu'allonger le sommeil : acquittement toutes les 2 min en LPL (déjà fait), balises CTP plus espacées, rapports de vie moins fréquents. Les nœuds de comptage et d'intrusion étant sur secteur d'après le plan, seuls les trois nœuds thermiques ont réellement besoin de LPL.

> **Un piège découvert grâce à ces mesures.** En LPL, l'émulateur de la radio CC2420 produit parfois des paquets incohérents (avertissement Cooja « packet size too large - repeating packet bytes ») dont le contrôle CRC est pourtant valide. Un acquittement ainsi altéré disait « tout est reçu » : les capteurs effaçaient des données jamais arrivées (taux de livraison tombé à 93 %). Depuis que la passerelle signe ses acquittements, ces faux messages sont rejetés (196 rejets `ACQUIT_REJETE` sur les essais d'énergie). Artefact de simulation ou brouillage réel, la leçon est la même : un message de contrôle doit être authentifié autant que les données.

Le taux de livraison des essais d'énergie est mesuré « à terme » (rapports publiés au moins 5 min avant la fin) : en LPL, un rapport perdu n'est renvoyé qu'au cycle d'acquittement suivant, 2 min plus tard.

5.7 Authentification (ENF6)

Les 10 rapports de l'intrus (même programme, autre clé) — 30 envois avec ses renvois, puisqu'il n'est jamais acquitté — sont tous rejetés (`REJET_AUTH`), et aucun rapport légitime ne l'est. Dans l'autre sens, un nœud sans la clé rejette les acquittements de la passerelle : l'intrus du test est volontairement programmé pour passer outre et émettre quand même. Le code d'authentification est calculé par le capteur et vérifié par la passerelle : un relais ne peut pas modifier un rapport sans être détecté.

---

6. Exigences vérifiées

EXIGENCE	TEST	VALEUR MESURÉE	VERDICT
EF1 — une mesure toutes les 30 s ± 2 s	T1 (10 min, 9 capteurs)	écart entre rapports : 30,00 à 30,00 s (175 intervalles)	✓ conforme
EF1 — valeur transmise sans altération	T1	61 temperatures comparees capteur/passerelle, 0 differentes	✓ conforme
EF2 + ENF1 — 95 °C → alerte < 10 s (cible 5 s)	T2 (presse M2, seuil 90 °C)	alerte 0,88 s après la mise à 95 °C	✓ conforme
EF2 — seuil propre à chaque machine	T2 (four M3, seuil 220 °C)	185 °C sans alerte ; 225 °C → alerte en 0,32 s	✓ conforme
EF5 — 100 pièces → compteur = 100	T4 (poste C1)	compteur local 100, reçu par la passerelle 100	✓ conforme
EF5 — le compteur survit à un redémarrage	T4 (poste C2, chien de garde)	40 pièces avant, 40 restaurées	✓ conforme
EF6 — présence hors 08h–18h → alerte < 5 s	T5 (P1, P2, P3 à 19h00)	alerte en 0,05 s au pire (3 zones), buzzer local activé	✓ conforme
B4 — pas d'alerte pendant les heures ouvrées	T5 (17h30 et 08h30)	0 alerte(s) pendant les heures ouvrees (presences a 17h30 et 08h30)	✓ conforme

EXIGENCE	TEST	VALEUR MESURÉE	VERDICT
ENF1 / T6 — latence totale < 10 s	T6 (20 min, 10 nœuds)	capteur → passerelle : moyenne 33,1 ms, max 56 ms	✓ conforme
ENF1 — budget nœud → passerelle < 1 s	T6	95 % des rapports en moins de 44 ms	✓ conforme
B3 — données centralisées (taux de livraison)	T6 / T6b	PDR 100,00 % (liens parfaits), 100,00 % (liens dégradés)	✓ conforme
ENF3 / T7 — coupure d'1 h : données gardées puis renvoyées	T7 (passerelle hors de portée 1 h)	1080 / 1080 rapports récupérés, tampons vidés en 114 s	✓ conforme
ENF3 — horodatage de la mesure conservé	T7	écart max 3,6 s (critère retenu : ± 5 s)	✓ conforme
ENF8 / T9 — un nœud retiré, les autres continuent	T9 (retrait du relais T1)	8/8 capteurs continuent, PDR après la panne 100,0 %	✓ conforme
ENF8 — reconvergence du routage	T9 (P1, qui passait par T1)	nouveau parent en 18,2 s, rapports rétablis en 47,3 s	✓ conforme
ENF8 — détection du nœud défaillant	T9	alerte NOEUD_MUET après 84 s	✓ conforme
ENF6 — messages authentifiés	ENF6 (11e nœud sans la clé)	30 rapports de l'intrus rejetés, 0 rapport légitime rejeté	✓ conforme
ENF5 — autonomie ≥ 6 mois	E_energie (PowerTracker)	radio toujours allumée : 99,5 % → 4 jours ; meilleure variante LPL : 71 jours (estimation, voir § 5.6)	✗ non conforme
ENF4 / T10 — disponibilité ≥ 99,5 %	—	non mesurable en simulation (pas de pannes matérielles aléatoires) ; voir § 6	—

ENF4 (disponibilité ≥ 99,5 %, T10) n'est pas mesurable en simulation : il faudrait des pannes aléatoires sur une longue durée. La simulation en donne les briques : détection des nœuds muets, reconvergence automatique (T9), stockage pendant les coupures (T7). **T3 et T8** (non rappelés dans la consigne) ne sont pas traités.

Détail de toutes les vérifications

JOURNAL	VÉRIFICATION	DÉTAIL (TEL QU'ÉCRIT PAR LE SCRIPT)	RÉSULTAT
`T1_periode`	T1	9 capteurs, 175 intervalles entre 30.00 et 30.00 s (tolerance 28-32 s)	✓ conforme
`T1_periode`	T1_fidelite	61 temperatures comparees capteur/passerelle, 0 differentes	✓ conforme
`T2_surchauffe`	T2	M2 a 95 C : alerte 0.88 s apres l'injection (exigence < 10 s, cible 5 s)	✓ conforme
`T2_surchauffe`	T2_json_status	JSON MACHINE_02 avec status ALERTE publie	✓ conforme
`T2_surchauffe`	T2_seuil_par_machine	M3 a ~185 C sans alerte (seuil 220 C), puis alerte a 225 C en 0.32 s	✓ conforme
`T2_surchauffe`	T2_fin_alerte	FIN_ALERTE SURCHAUFFE MACHINE_02 apres retour a la normale	✓ conforme

JOURNAL	VÉRIFICATION	DÉTAIL (TEL QU'ÉCRIT PAR LE SCRIPT)	RÉSULTAT
`T4_comptage`	T4	100 pieces injectees : compteur local=100, production_count recu=100	✓ conforme
`T4_comptage`	T4_redemarrage	C2 redemarre a 40 pieces, RESTAURE compteur=40, passerelle voit REDEMARRAGE puis production_count=42	✓ conforme
`T5_intrusion`	T5	alertes de nuit : ZONE_01=0.05 s ZONE_02=0.05 s ZONE_03=0.05 s (exigence < 5 s, buzzer local actif)	✓ conforme
`T5_intrusion`	T5_pas_alerte_de_jour	0 alerte(s) pendant les heures ouvrees (presences a 17h30 et 08h30)	✓ conforme
`T6_reseau`	T6	latence capteur->passerelle max 56 ms, moyenne 33.1 ms sur 360 rapports (exigence < 10 s)	✓ conforme
`T6_reseau`	T6_budget_radio	95 % des rapports en moins de 44 ms (budget noeud->passerelle < 1 s)	✓ conforme
`T6_reseau`	T6_pdr	taux de livraison 100.00 % (seuil 99 %)	✓ conforme
`T6_reseau`	T6_multisaut	P1, P2, P3 hors de portee du puits : sauts = 2, 2, 2	✓ conforme
`T6b_pertes`	T6b	latence capteur->passerelle max 116 ms, moyenne 37.2 ms sur 360 rapports (exigence < 10 s)	✓ conforme
`T6b_pertes`	T6b_pdr	taux de livraison 100.00 % (seuil 98 %)	✓ conforme
`T7_coupure`	T7	rapports publies pendant 1 h de coupure puis recus : 1080/1080 (2:120/120 3:120/120 4:120/120 5:120/120 6:120/120 7:120/120 8:120/120 9:120/120 10:120/120)	✓ conforme
`T7_coupure`	T7_horodatage	ecart max entre timestamp JSON et instant de mesure : 3.6 s (critere +/- 5 s = 1/6 de la periode)	✓ conforme
`T9_panne`	T9	T1 retire : 8/8 autres capteurs continuent, PDR apres panne 100.0 %	✓ conforme
`T9_panne`	T9_reroutage_P1	P1 passe par 9 (3 sauts), rapports retablis en 47.3 s	✓ conforme
`T9_panne`	ENF8_detection	passerelle signale NOEUD_MUET T1 apres 84.5 s	✓ conforme
`ENF6_auth`	ENF6	30 rapports de l'intrus rejetes, 0 acceptes ; 0 rapport legitime rejete	✓ conforme
`E_energie`	E	duty cycle radio mesure sur 10 noeuds, moyenne capteurs 99.50 %, PDR 100.00 %, latence moy 33 ms	✓ conforme
`E_energie_lpl`	E_lpl	duty cycle radio mesure sur 10 noeuds, moyenne capteurs 5.64 %, PDR 100.00 %, latence moy 392 ms	✓ conforme
`E_energie_lpl1024`	E_lpl1024	duty cycle radio mesure sur 10 noeuds, moyenne capteurs 11.00 %, PDR 100.00 %, latence moy 1040 ms	✓ conforme
`E_energie_lpl2048`	E_lpl2048	duty cycle radio mesure sur 10 noeuds, moyenne capteurs 20.74 %, PDR 100.00 %, latence moy 2224 ms	✓ conforme

7. Limites et perspectives

- **Prototype réel ESP32 + MQTT.** Les lignes de la passerelle contiennent déjà le topic et le JSON : un pont série → `mosquitto_pub` suffit pour alimenter Node-RED, InfluxDB et Grafana. En WiFi, CTP disparaît (le point d'accès joue la racine) mais le tampon et l'acquittement QoS 1 restent nécessaires.
- **Sécurité.** Le code d'authentification sur 16 bits est pédagogique (FNV-1a « salé »), sans protection contre le rejeu ni chiffrement. En réel : AES-CCM matériel du CC2420 (802.15.4) ou TLS sur MQTT (port 8883) avec un certificat par ESP32, et un compteur anti-rejeu.
- **Énergie.** Combiner LPL, une période de rapport adaptée (30 s seulement pour la température, quelques minutes pour les rapports de vie) et un intervalle de balises CTP plus long. Les nœuds de comptage et d'intrusion sont sur secteur (plan) : seuls les nœuds thermiques ont besoin de LPL.
- **Stockage 24 h.** Passer le tampon en flash externe (LogStorage) pour couvrir ENF3 en entier.
- **Heure d'usine.** Chaque nœud la calcule depuis son démarrage (écart ≤ 1 s entre nœuds dans Cooja). En réel : NTP sur l'ESP32, ou l'heure de la passerelle diffusée par Drip.
- **Modèle radio.** UDGM est un disque parfait. Une étude plus réaliste utiliserait le modèle MRM ou DGRM de Cooja avec les murs du plan.

Généré le 21/10/2026 par `rapport/generer_rapport.py` à partir de `resultats/*.log`. Pour tout relancer : `bash tout_relancer.sh` dans WSL, puis `python rapport/generer_rapport.py`. Code, scénarios et guide : `simulation/02_simulation_projet/README.md`.